INSTITUTE OF INFORMATION TECHNOLOGY
UNIVERSITY OF DHAKA

IIT
University of Dhaka

StateEye

An intelligent web-testing tool inspired by the FRAGGEN approach

Technical Report

SE - 801 - Final Project

Supervised by:

Mridha Md. Nafis Fuad
Lecturer
Institute of Information Technology
University of Dhaka

Submitted by:

Mosamma Sultana Trina (1313)
Institute of Information Technology
University of Dhaka

Submitted to:

SPL 3 Program Committee

Date of Submission

26th February, 2026

Letter of Transmittal

26th February, 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **StateEye** (An intelligent web-testing tool inspired by the FRAGGEN approach)

Sir,

I am submitting the technical report for our Software Project Lab 3, "**StateEye**: An intelligent web-testing tool inspired by the FRAGGEN approach".

In this report, the technical details (design, implementation and testing approach) of this software project have been provided for you to review. Although I have made an effort to provide the best report I can, I would appreciate any comments you may provide to help me improve my work.

Thanks again for your time and consideration.

Sincerely,
Mosamma Sultana Trina
BSSE-1313
Institute of Information Technology
University of Dhaka

Supervisor's Signature

Acknowledgement

Thanks to the Institute of Information Technology (IIT) for the chance to work on the "StateEye" (An Intelligent Web Testing Tool) project via the FragGen approach. I would also like to offer my deepest appreciation and respect to my supervisor, Mridha Md. Nafis Fuad, Lecturer at IIT, University of Dhaka for his direction and support throughout this project. I greatly respect his vast experience and all that he has done to help guide me down the path I had taken thus far and will continue along in doing work with regard to web testing.

Abstract

Frequent changes in modern web applications make functionality better. Unfortunately, they can also cause unintentional regressions, which can be especially difficult to find through manual testing methods due to speed and scalability limitations of testing. In addition, traditional automated testing methods (which are based on whole page comparisons) allow for some types of near-duplicate states to go undetected.

StateEye will solve this problem by automatically exploring an application and building a graph-based model of how it behaves. Instead of comparing the page as a whole, StateEye will break the page down into fine-grained fragments so that it may accurately identify states that are either a clone, near duplicate, or unique. As a result, the integration of fragment-level analysis will decrease redundant explorations and increase the reliability, performance, and scalability of web application testing through generating strong regression tests that can withstand visual changes.

Table of Contents

List of Figures

List of Tables

1. Introduction

1.1 Background

Modern web applications change frequently. Even small updates can lead to unexpected issues. Regression testing is often done manually or with record-and-replay tools, which can be expensive and unreliable when the user interface changes. While automated model-based testing can help, current methods depend on comparing entire pages. This approach makes them ineffective at dealing with near-duplicate but functionally identical states. As a result, it leads to redundant models and weak test oracles.

1.2 Motivation

The motivation for this project comes from the weaknesses of current automated web testing methods in dealing with modern, dynamic web applications. Whole-page comparison techniques are very sensitive to minor changes in layout, data, or styling. This creates frequent false positives during regression testing. As a result, automated test suites become unreliable and need a lot of manual intervention. The FRAGGEN research showed that treating a web page as a single unit is fundamentally flawed. Web pages consist of multiple functional fragments that may be reused or duplicated across different states. Analyzing these fragments allows for more accurate identification of functional equivalence and supports effective regression testing. StateEye aims to turn these research findings into a practical engineering system that works in the real world. By using fragment-based state abstraction, graph-based modeling, and automated test oracle generation, StateEye hopes to lessen the effort needed for regression testing while improving test reliability and coverage.

1.3 Problem Statement

Common issues present in automated web testing tools that currently exist include:

- A reliable way to identify when two applications have very different states even if they are functionally similar,
- Repeatedly testing functionally equivalent pages (not testing each page uniquely),
- The design of test oracles which are sensitive to minor UI change or data change,
- The excessive cost of maintaining an automated regression suite of tests.

There is an increasing demand for a smarter web testing system that will provide a higher level of accuracy in modeling web application behavior; provide for testing that distinguishes between meaningful functional differences and simply changing the look of an application; generate valid regression tests automatically.

1.4 Objectives

The goals of the StateEye project include:

- Automatic discovery and modeling of web application behavior through the use of fragmented state abstraction.

- Finding clone, near-duplicate, and separate states while not relying on manually set similarity threshold levels.

- Producing reusable and reliable regression test cases based on inferred application models.

- Allowing for small layout and data changes to be accommodated while accurately detecting true functional regressions.

- Assisting manual testers by giving them visual feedback and guiding their navigation through explorer.

1.5 Scope of the Project

StateEye is focused on providing automated regression testing for web applications. The StateEye regression testing solution is built to:

• Run against the client-supported web application using a browser automation framework

• Analyze dynamic DOM structure and visual fragments of a web page

• Automatically generate test cases and test oracles as output of the regression tests

• Support cross-version regression testing for all versions of the tested web application

The primary intent of the development of this project is not to completely replace the use of any form of manual testing and does not replace the use of semantic reasoning for any business rule logic in the application's definition. Testing for server-side business rule logic and for security of the application will not be covered by this work.

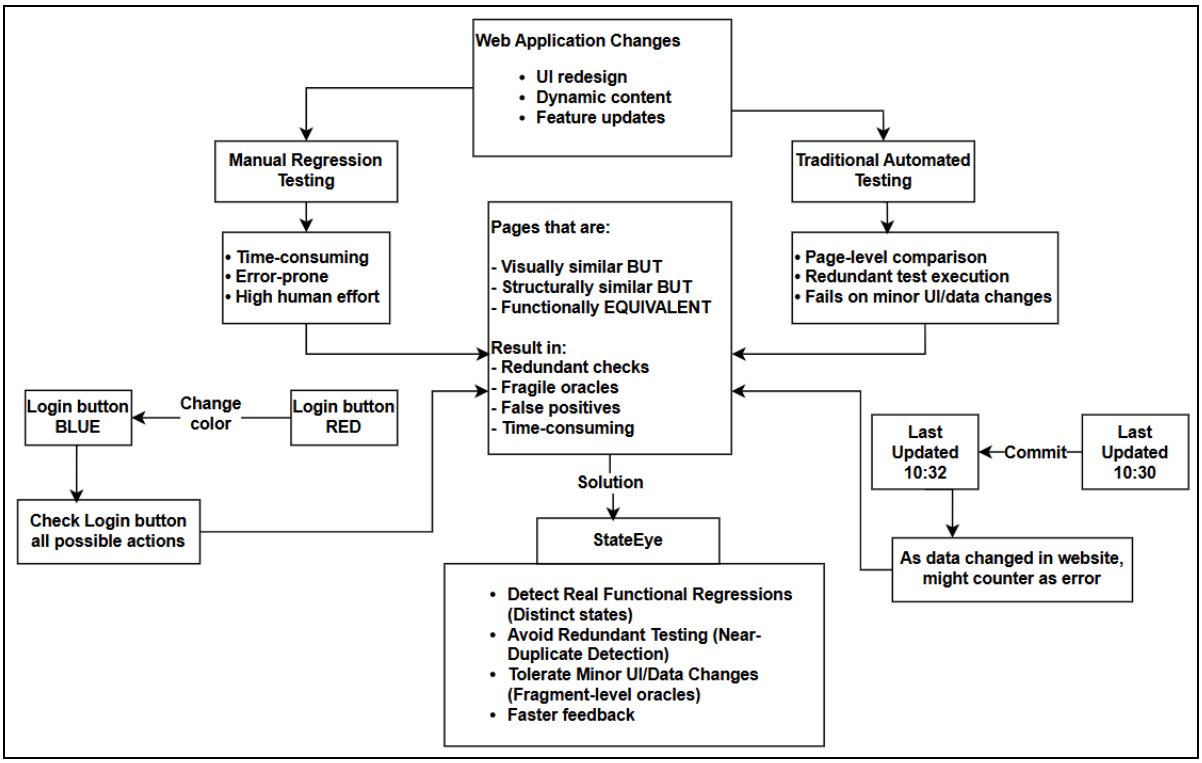


Fig-1: Problem Scenario

1.6 Timeline

A 4-month timeline has been planned for this project.

Task	October	November	December	January	February
Prerequisite Knowledge	_____				
Project Initialization	___				
Proposal Writing	__				
SRS Documentation	__	_____			

Task	October	November	December	January	February
Prerequisite Knowledge	_____				
Project Initialization	___				
Tool Design		_____	__		
Tool Development		_____	_____	_____	
Tool Testing				_____	__
Final Report Writing					_____
Final Demonstration					____

Table-1: Timeline for StateEye

1.7 Purpose of This Document

This document includes an in-depth description of the StateEye system, functional and nonfunctional characteristics of the StateEye system, systems models associated with the StateEye system, architectural design of the StateEye system, and testing of the StateEye system. This document's purpose is to provide a reference for use by developers, evaluators, and academicians who will be developing, assessing, or evaluating the StateEye system project.

2. Project Description

The foundation for the creation of StateEye was to compile a review of existing literature regarding automated testing online, regression techniques and model based testing for dynamic web applications. The research focused heavily upon an approach known as Fragment-based Test Generation for Web Applications (FRAGGEN), which is a methodology that demonstrates the inadequacies present when comparing a complete webpage to identify changes in behavior between the two versions; therefore many changes will not be found. Additional studies performed on web crawling, DOM based analysis, and state abstraction provided a wealth of information relating to how to handle near duplications of application states and the weakness of test oracles (tests that have a known answer).

The foundation of this research was augmented by reviewing problems that technology testers and developers encounter while validating web applications that are continually updated. Problems such as redundant execution of tests, false positives caused by cosmetic changes to the user interface (UI) and the cost associated with maintaining a large regression suite were all examined to identify possible causes of these problems. These findings provided the motivation for designing StateEye, which is a fragment aware, graph based testing system that looks for changes in functional behavior not simply on a change in how the page looks like after it has been updated.

To model and test for accuracy, StateEye uses CRAWLJAX to crawl the web and generates a representation of an application's behavior in the form of a graph by providing fragment level abstraction of the web pages being watched. Using the constructed graph, StateEye is able to automatically generate regression test cases.

2.1.1 Normal Requirements

Normal requirements refer to the basic functions and purposes of StateEye that must be met in order for StateEye to work as an automated web testing tool. Full compliance with these requirements will ensure that the StateEye system has minimum usability and overall effectiveness.

Functional Requirements:

- Fully Automated Crawling of Web Sites: The system will crawl through web applications and will trigger user events by clicking on links and buttons.
- Modeling of the State of the Web Application: The system will create a graph representation of the state of the web application by tracking the response from the application after the user has clicked on an event.
- Fragmenting of Web Pages for Testing: The system will take the web page that it crawls to and break it into smaller pieces, or fragments, in order to allow the StateEye system to analyze the page.
- Regression Test Case Creation: The system will create reusable regression test cases based on the application model inferred from the state of the web application.

Non-Functional Requirements:

- Usability: The StateEye system will require little to no configuration from the user and can complete an automated crawl without requiring any input from the user.

2.1.2 Expected Requirements

Expected requirements are those associated with an advanced automated web testing system that are not necessarily described in detail, but the lack of one or more of the expected requirements could be detrimental to the user's overall level of satisfaction with the system once they begin using it.

Functional Requirements:

- Cloning/Near Cloning of Application States: The StateEye system will identify the same, or very similar, application states using a fragment by fragment comparison.
- Avoidance of Redundant Exploration: The StateEye system will not re-explore application states that are identical to states previously explored.
- Robust Test Oracles: The StateEye system will generate test oracles that are able to tolerate minor differences in layout and/or structure.

2.1.3 Exciting Requirements

StateEye has more than standard functional requirements, advanced options that provide user experiences that differ from the traditional means of web testing.

Functional Requirements:

1. Automated and Manual Testing Modes: This system should have two testing modes, automated and manual-assisted.

2. Manual Testing Support: In manual-testing mode, this system should enable the manual-user to interact with the web application while simultaneously tracking their current and all previously visited states and actions.

3. Recommended Exploration: This system should recommend previously untested buttons and links or untested states to encourage a tester to explore previously untested areas of the application.

Non-Functional Requirements:

1. Response to User Actions: The visual feedback of both previously-tested and untested areas should be updated within a reasonable time, so users will not notice any lag time.

2.2.1 Usage Scenario for StateEye

StateEye offers both complete autonomy for automated/tester-independent testing via intelligent support, and assistance to testers through the use of automated/manual test scenarios. Below is a list of how the system will be utilized.

- A tester runs StateEye from their local environment. The tester enters the URL of the target web application, and the StateEye application will create a new browser session to open that URL.
- Selecting Testing Mode
- Before testing can commence, the tester will select which testing method (testing approach) is to be used:

Automated Testing (Tester-free exploration): The tester will run an automated test against a web application. In this mode, StateEye will automatically generate a test case by recording user interaction and building the state graph as it goes.

Manual Testing (Assisted by StateEye): The tester will run a manual test against a web application while StateEye is running. In this mode, StateEye will monitor the user's actions and provide real-time assistance on how to continue performing manual

Automated Exploration and Fragment Classification

StateEye explores the web application automatically via clicking on links or buttons in so-called ‘automated test mode’. At each visited page, it divides the page into fragments. It performs a fragment-level computation to determine if the two fragments are in the same state. If both fragments are determined to be in the same state, they can be identified as clones and/or near clones to reduce duplicate exploration costs.

Manual Testing with Live Feedback

When testing the web application manually in ‘manual assisted test mode’, the tester is free to navigate through the web application. When the tester clicks a button or navigates to another state, StateEye shows the tester which areas of the web application have or have not been tested. Untested buttons or fragment will be highlighted to enable the tester to make an informed decision about where to navigate next.

State Modeling & Test Creation

In both automated and manual testing modes, StateEye constructs an application behaviour model (graph-based). According to the application behaviour model constructed above, StateEye will generate automatically generated regression test cases and the appropiate test oracles.

Regression Verification

The generated regression tests will be run on newer versions of the web application to validate re: functional correctness between versions. If minor cosmetic changes exist between test versions, they will not be reported as regression failures but rather, if there are any major functional regressions, those will be reported.

2.2.2 Example Scenario Walkthrough

User: Software Tester Performing Validation of E-commerce Website Update on Software.

Execute Tool: The Tester Executes StateEye's software by entering E-Commerce Website URL.

Select Type: The Tester selects to perform Manual Assisted Testing Type.

Performing Actions: The Tester clicks on product categories and navigates by clicking on buttons throughout the website.

Real Time Support: Each button/picture visited by the Tester, StateEye will mark the state and interface fragment as Tested. StateEye will also indicate untested buttons/pictures on the website.

Guided Exploration: StateEye will provide suggestions for untested buttons/links to assist the Tester with efficient and non-redundant exploration of the website.

State Graph Generation: The computer will update the testing state graph as the Tester performs actions in the user interface.

Test Case Creation: At the completion of the testing, StateEye will automatically generate test cases to return to for regression testing and track all the Manual and Automatic testing paths created.

Result: The Tester will complete testing with higher testing coverage and less work. Cosmetic changes of User Interface components will not be reported; however, functional regressions may be detected (broken navigation links or incorrect transitions between user interface states).

3. Scenario-Based Modeling

3.1 Use Case Diagram

A use case diagram is a graphical illustration in software engineering that describes how the system will interact with each of its external entities (or actors) to accomplish a specific set of objectives or tasks. The use case diagram demonstrates all relationships between the various use cases (the functionalities of the system) and the actors (the external user or system interacting with the functionality of the system). Use case diagrams are traditionally created during the requirements-gathering phase of a project, where the system's functionality and user interactions are gathered, which will be mapped out as part of the use case diagram using applicable software modeling tools.

3.1.1 StateEye

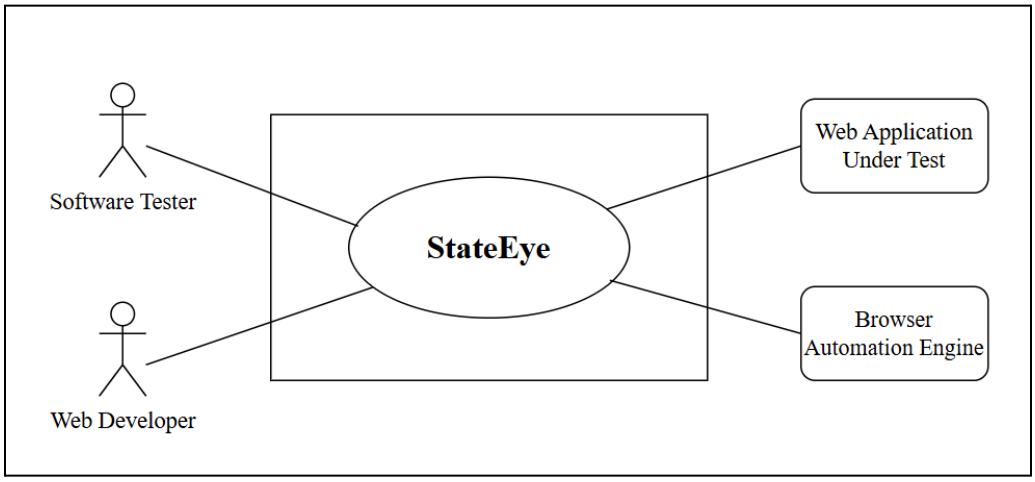


Fig-2: Use Case Level 0 (StateEye)

Primary Actors:

- Software Tester
- Web Developer

Secondary Actors:

- Web Application Under Test
- Browser Automation Engine

Goal in Context:
The goal in this context is shown in the diagram above and relates to the way in which the tester and developer are able to use the fragment based automated regression testing system called StateEye and how StateEye interacts with the Web application using the browser automation engine to verify that there is no change in behavior of the application after an update.

3.1.2 Modules of StateEye

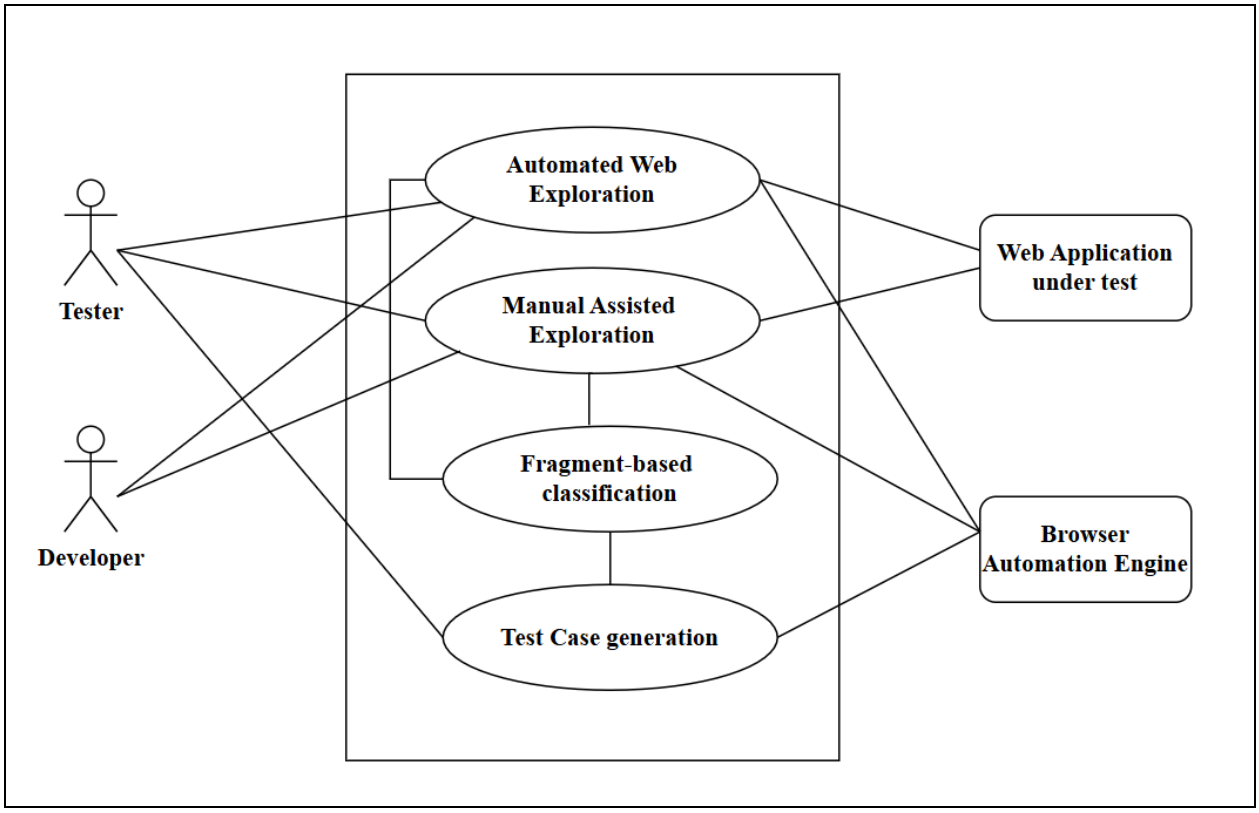


Fig-3: Use Case Level 1 (StateEye)

Primary Actors:

- Software Tester
- Web Developer

Secondary Actors:

- Web Application Under Test
- Browser Automation Engine

Goal in Context:

The use cases can be elaborated as follows:

3.1.2.1 Automated Web Exploration

The Automated Web Exploration module is defined as part of the crawler driven exploration phase of the StateEye system, and is based upon the automated exploration mechanism of the FRAGGEN approach. StateEye utilizes this mode to autonomously explore the Web Application Under Test by identifying and executing user actions (e.g. clicking elements, following links)

that may be executed by a user at each current state of the application. Each action ' ' executed results in a state transition between the current state and the new state of the application, or e=(si,a,sj) i.e. si is the current source state, sj is the new target state. All of the states and transitions that have been executed will comprise of the application graph defined as G = (S,E) where the set of all discovered states is denoted as 'S', while the set of all transitions, between discovered states, is denoted as 'E'

The Browser Automation Engine is responsible for executing the actions that StateEye determines to execute, and for capturing the DOM and creating visual captures (snapshots) of the Web Application Under Test after each action has been executed. For each newly discovered state, StateEye will decompose the corresponding DOM into fragments and create a fragment based equivalence relation to determine if the state is a clone, nearly clone, or distinguishable from the current state. Thus, this classification of states prevents redundant state exploration and mitigates uncontrolled growth of the states that comprise the application graph.

3.1.2.2 Manual Assisted Exploration

The Manual Assisted Exploration module of FRAGGEN allows testers or developers to discover states through manual exploration based on formal state-transition modeling principles. Manual exploration is different from automated exploration as the interaction and exploration will be done by a human. The Software Tester or Web Developer can click on buttons, go to different pages, input data into forms, etc., to perform an action (a), which is the state-transition action (a) defined in the state-transition model; when a response is given to the performed action (a) it creates a new state (sj). Therefore, the manual exploratory process creates transitions in the state-transition model as follows:

$$e = (si, a, sj)$$

These transitions are added to the same graph representation (G = {S, E}) created through automated exploration. To provide support for effective manual testing, StateEye applies the fragment-based classification model to the states being observed in real time, and produces visual markings of both the explored and unexplored fragments and interactions. This will help testers concentrate on parts of the application that have not been explored while ensuring that both the model being constructed and the equivalence logic within the models have not been changed.

3.1.2.3 Fragment-based Classification

The fragmentation-based classification system is the heart of the FRAGGEN project. Instead of matching complete web pages, StateEye extracts UI fragments from each DOM it sees; that is, it decomposes the DOM into a collection of logical UI portions, with each logical UI portion representing a semantic break in the way that part of the user interface is intended to function.

For each new DOM state represented by sj (state sj has been observed for the first time), StateEye compares the previously observed UI fragment information with previously encountered UI fragment information to determine whether state sj should be classified as a Clone state (both the structural and the content representations are identical), a Near-duplicate state (structurally similar, but with some minimal change to the data within the content representation or some other limited layout difference), or a Distinct state (a functional behavior that has not previously been observed or recorded). The classification (i.e., the classification of state sj) will determine whether the Clone and certain Near-duplicate nodes are added (as new nodes) to the state set SSS or any Distinct state extends the state set S. Thus, the net result of the functionality of the fragment level will be to provide precise identification of meaningful functionality and to eliminate false positives based on minor differences in either the user's display of the user interface or the data presented by the user interface.

3.1.2.4 Test Case Generation

The Test Case Generation component implements the fragment-based test generation and oracle construction models defined in the FRAGGEN method. The test cases are created using the state graph 'G', which was created from the test model. Each test case is based on a path from the initial state 's0' to the last state 'sn', using a sequence of actions (ai, where 'i' starts at 1 and ends at n) along the way.

At each of the states of the path, StateEye creates the test oracles for the fragments of the UI. The expected fragments and their structure and relationship to other fragments will be documented in such a way as to allow for the creation of tests that will not be affected by any cosmetic changes to the UI but will continue to find significant regressions in behavior.

All information necessary to create the tests such as trace actions, DOM images, fragment maps, state classifications, are tracked during the exploration phase of the Browser Automation Engine.

4. Activity Diagram

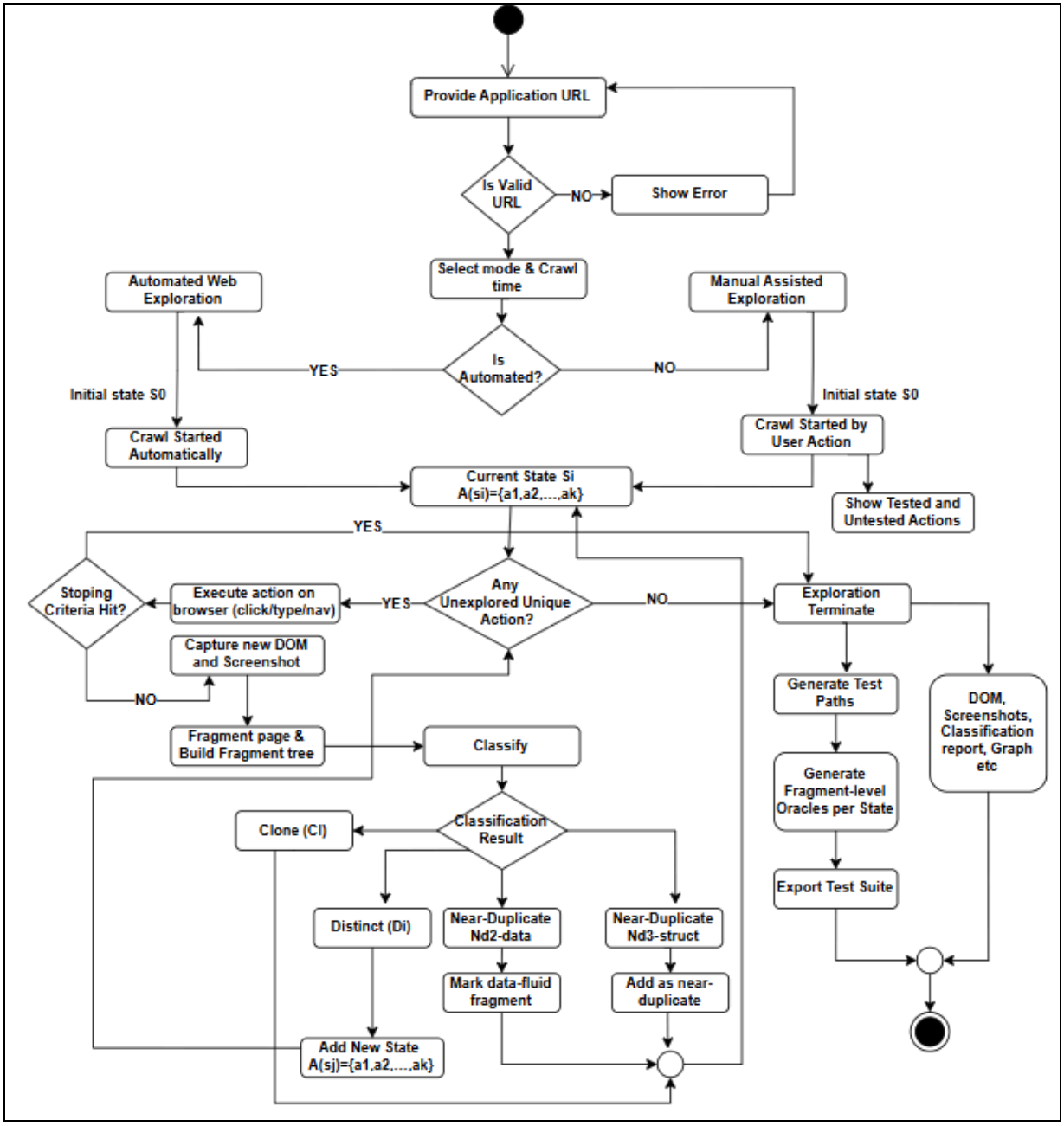


Fig-4: Activity Diagram of StateEye

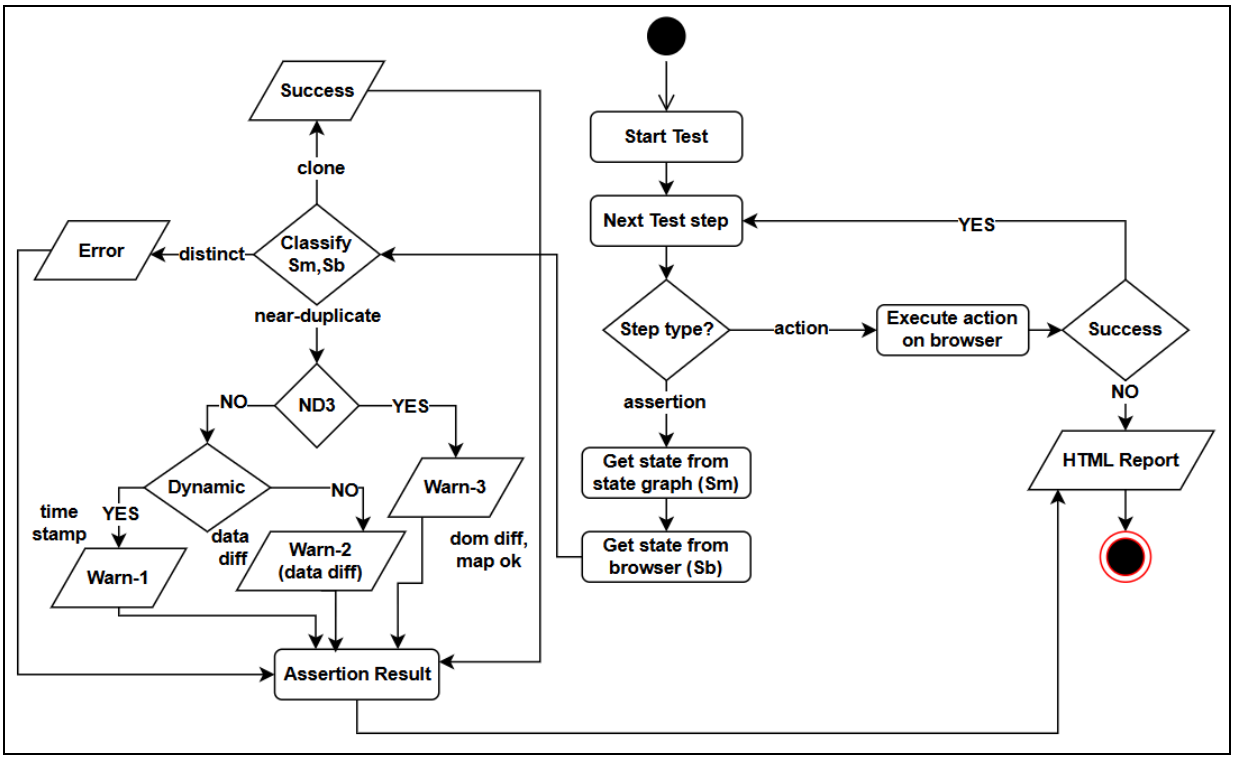


Fig-5: Test Execution

5. Data based Modeling

StateEye's data is stored in a Document Store as a collection of crawls, or CrawlSession documents that contain a metadata section and a set of references (paths) to State content stored in directories of HTML, DOM, screenshot, graph, and classification.

Rule of Storage: Store Big Files as Files and Use JSON to Store References; See Document Store Diagram below.

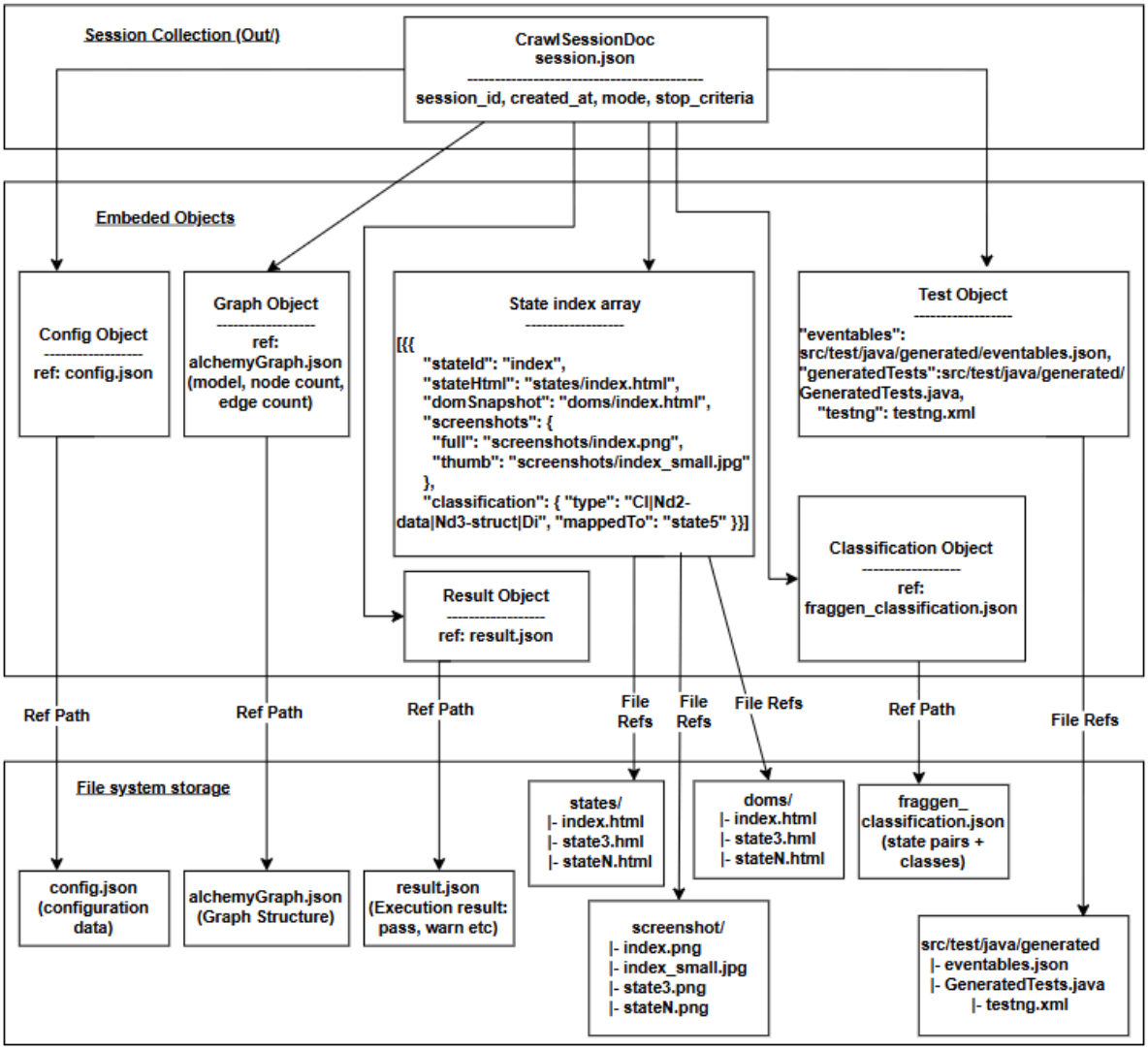


Fig-6: Data based Modeling

6. Class-Based Modeling

Modeling in the class-based style is an essential part of designing and building object-oriented systems. Modeling in this way allows for visual representations of all of the elements that comprise the software being built and how each element interacts with other elements. An example of modeling with a class-based approach can be seen below.

6.1 Analysis Classes

For my solution domain analysis, I initially began by identifying all of the nouns in the scenarios I provided. Next, I used the General Classification to filter out those nouns that I felt belonged in the solution domain (external entities; things; events; roles; organizations; locations; structures) and which I believed to be, at minimum, a class in my analysis. I then used selection criteria (retained information, required services, multiple attributes, common attributes, common operations, essential requirements) in vetting what nouns would be included in my analysis of classes.

Based on the description given for the StateEye system, my analysis was based on the following identified classes:

1. URL / Web Application
2. Crawl Session
3. State
4. DOM Snapshot
5. Screenshot
6. Fragment
7. Fragment Tree
8. Actionable Element
9. Event
10. Transition
11. State Graph
12. Fragment Classification
13. clone
14. near-duplicate
15. distinct
16. Memoization Map
17. Data-fluid Fragment
18. Automated Exploration
19. Manual Assisted Exploration
20. Test Case
21. Test Oracle

22. Test Suite
23. Test Result
24. Crawl Session
25. Browser Automation Engine

We have consolidated similar responsibilities from different classes and created a Class Responsibility Collaborator Diagram showing the combined classes.

Derived Classes and Final Grouping -

1. Crawl Session
2. State
3. DOM Snapshot
4. Screenshot
5. Fragment
6. Actionable Element
7. Event
8. State Graph
9. Memoization Map
10. Data-fluid Fragment
11. Test Suite
12. Test Result

6.2 CRC Cards

Crawl Session	
Attributes	**Methods**
-sessionId -baseUrl -mode -startTime -endTime -status -outputFolderPath -configRef -stateGraphRef -classificationRef	+ startSession(baseUrl, mode) + stopSession(reason) + loadConfig() + saveArtifacts() + getSessionSummary()

-testSuiteRef -testResultRef	
Responsibilities	**Collaborators**
● Initialize and manage one complete run of StateEye. ● Store references to all generated artifacts (graph, states, classification, tests, results). ● Track run status, timing, and output location for reproducibility	● State Graph ● State ● Test Suite ● Test Result

Table-2: CRC Card (Crawl Session)

State	
Attributes	**Methods**
- stateId - url - discoveredOrder - equivalenceClass - domSnapshotRef - screenshotRefs - rootFragmentRef	+ setEquivalenceClass(label) + attachDOMSnapshot(domSnapshot) + attachScreenshot(screenshot) + getRootFragment() + listActionableElements()
Responsibilities	**Collaborators**
● Represent a discovered web application state. ● Maintain structural and visual state data. ● Act as a node in the state graph.	● DOM Snapshot ● Screenshot ● Fragment ● State Graph

Table-3: CRC Card (State)

Dom Snapshot

Attributes	**Methods**
- domId - stateId - domHtmlPath - domHash - capturedAt	+ captureFromBrowser() + computeHash() + extractDOMTree()
Responsibilities	**Collaborators**
● Store DOM structure of a state. ● Support fragment extraction and structural comparison.	● Fragment ● State

Table-4: CRC Card (Dom Snapshot)

Screenshot	
Attributes	**Methods**
- screenshotId - stateId - imagePath - imageType - capturedAt	+ captureImage() + generateThumbnail() + compareVisual(otherScreenshot)
Responsibilities	**Collaborators**
● Store visual representation of a state. ● Support visual comparison and reporting.	● State ● Fragment ● Test Result

Table-5: CRC Card (Screenshot)

Fragment	
Attributes	**Methods**

- fragmentId - stateId - parentFragmentId - domNodeSignature - boundingBox - childrenFragments - equivalenceLabel - isDataFluid	+ buildFromDOM(domSnapshot) + compareFragment(otherFragment) + markDataFluid(flag) + getChildren()
Responsibilities	**Collaborators**
● Represent a functional UI fragment. ● Support fragment-level abstraction and comparison.	● State ● DOM Snapshot ● Screenshot ● Memoization Map

Table-6: CRC Card (Fragment)

Actionable Element	
Attributes	**Methods**
- actionableId - stateId - elementType - targetXpath - cssSelector - owningFragmentRef - exploredFlag	+ locateInDOM(domSnapshot) + triggerAction() + setExplored(flag)
Responsibilities	**Collaborators**
● Represent an interactable UI element. ● Enable automated and manual exploration.	● State ● Fragment ● Event

Table-7: CRC Card (Actionable Element)

Event	
Attributes	**Methods**
- eventId - stateId - actionType - targetLocator - triggeredBy - timestamp	+ recordEvent(actionableElement) + replayEvent()
Responsibilities	**Collaborators**
● Record interactions during exploration. ● Drive state transitions in the model.	● Actionable Element ● State ● State Graph

Table-8: CRC Card (Event)

State Graph	
Attributes	**Methods**
- graphId - sessionId - rootStateId - states - transitions - nodeCount - edgeCount	+ addState(state) + addTransition(fromState, event, toState) + exportGraph()
Responsibilities	**Collaborators**
● Maintain the inferred application model. ● Support path-based test generation.	● Crawl Session ● State ● Event ● Test Suite

Table-9: CRC Card (State Graph)

Memoization Map	
Attributes	**Methods**
- mapId - sessionId - uniqueFragments - duplicateMappings	+ registerFragment(fragment) + findDuplicate(fragment)
Responsibilities	**Collaborators**
● Reduce redundant fragment comparisons. ● Maintain unique fragment mappings.	● Fragment ● Crawl Session

Table-10: CRC Card (Memoization Map)

Data-fluid Fragment	
Attributes	**Methods**
- markerId - fragmentId - confidenceScore - reason	+ detectVolatility(fragment) + adjustSeverity()
Responsibilities	**Collaborators**
● Identify dynamic UI regions. ● Reduce false regression warnings.	● Fragment ● Test Result

Table-11: CRC Card (Data-fluid Fragment)

Test Suite	
Attributes	**Methods**
- suiteId	+ generateFromStateGraph(stateGraph)

- sessionId - testngXmlPath - generatedTestsPath - testCount	+ buildTestCases() + runSuite()
Responsibilities	**Collaborators**
● Generate executable regression tests. ● Manage test execution workflow.	● State Graph ● Event ● Test Result

Table-12: CRC Card (Test Suite)

Test Result	
Attributes	**Methods**
- resultId - sessionId - suiteId - passedCount - failedCount - warningCount - resultJsonPath	+ recordPass(testCase) + recordFailure(testCase) + exportResult()
Responsibilities	**Collaborators**
● Store test execution outcomes. ● Provide regression feedback.	● Test Suite ● Screenshot ● Fragment

Table-13: CRC Card (Test Result)

6.3 CRC diagram

The CRC diagram of the classes:

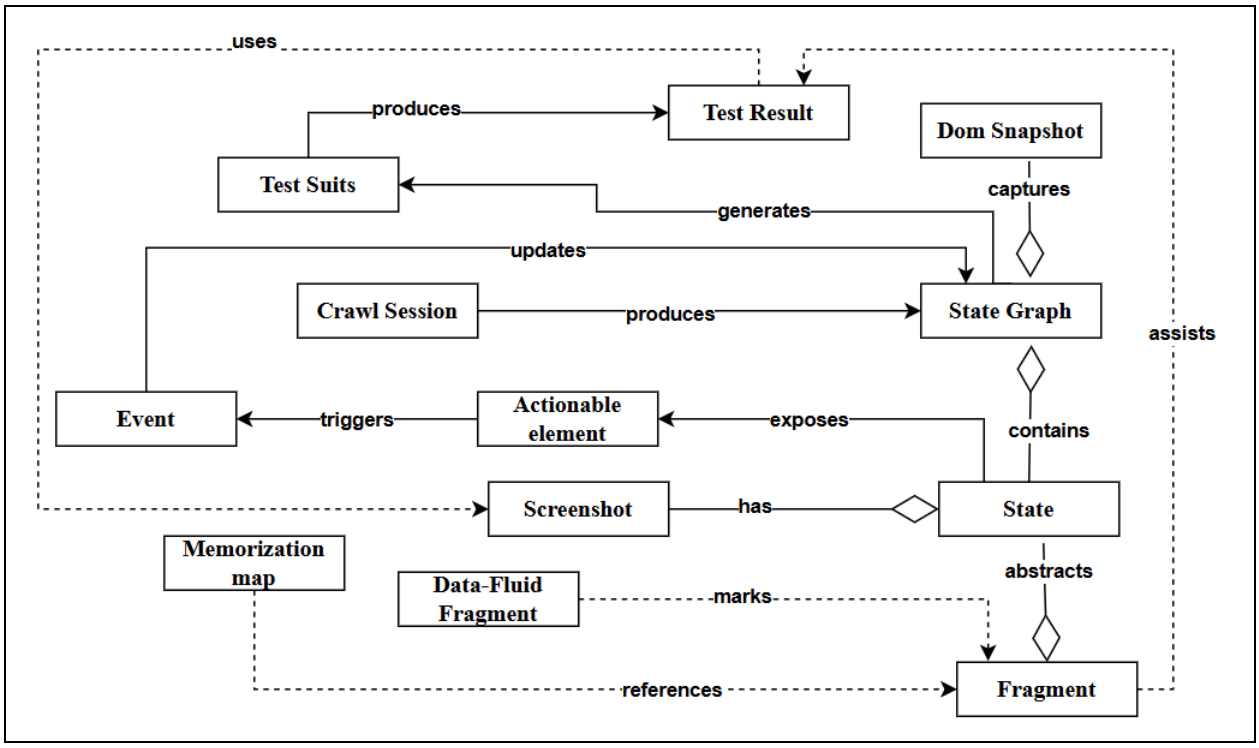


Fig-7: CRC diagram of StateEye

7. Architectural Design

The Architectural Design component is a graphic creation used in software engineering to outline the high-level structure and arrangement of the software system.

7.1 Architectural Context Diagram

Architectural context of the tool is in the following diagram-

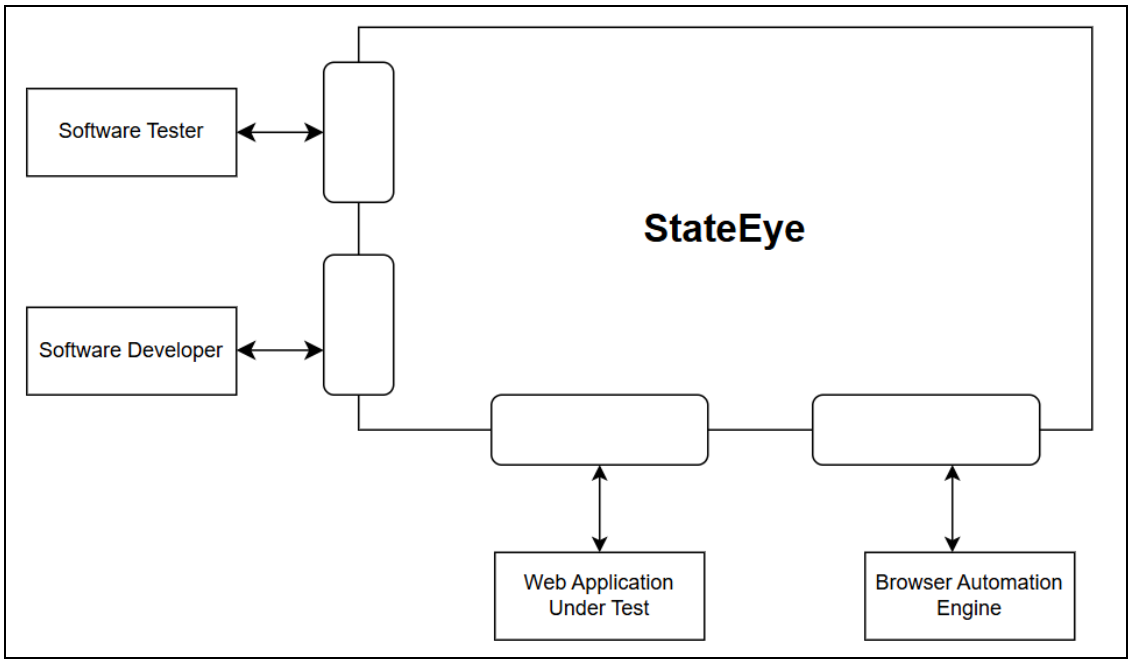


Fig-8: Architectural Context Diagram of StateEye

This is the architectural context diagram for StateEye, a web testing and regression analysis tool that provides intelligent testing capabilities for users in two primary user groups and supports integration with sub-systems in the architecture. Software testers utilize StateEye to validate web applications through automated/manual assisted testing. Software developers utilize StateEye to confirm recent code changes, detect unexpected regressions to web applications, and provide feedback regarding application behavior prior to deploying the applications.

7.2 Archetypes

The archetypes for the tool are defined below-

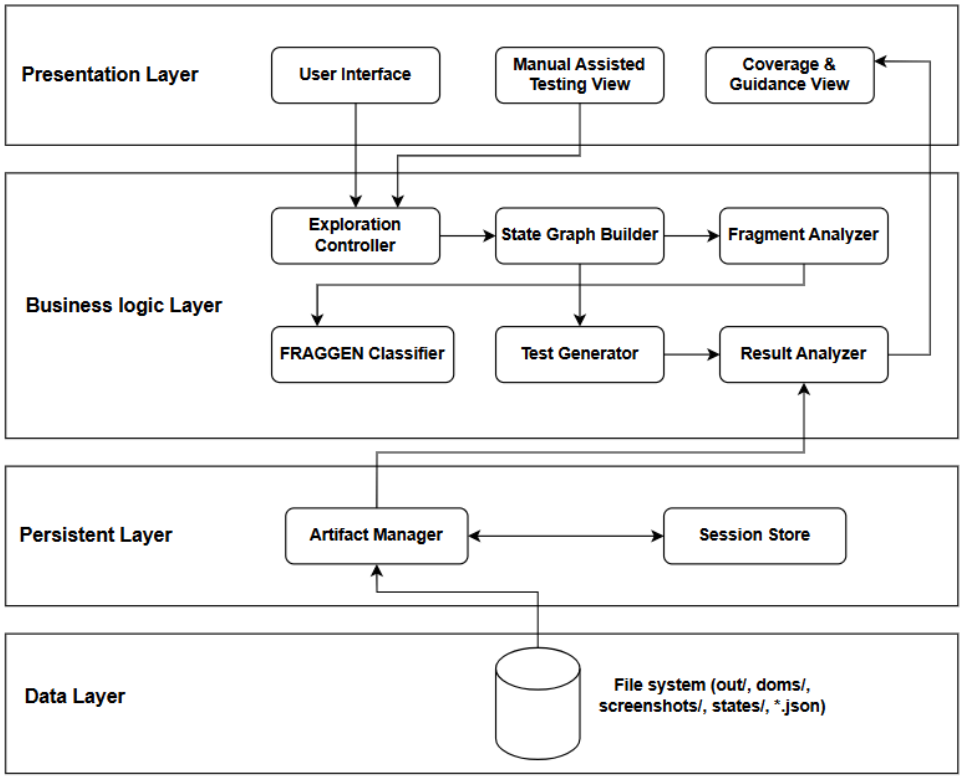


Fig-9: Archetypes of StateEye

1. Data Layer

File System: A centralized place to store all the persistent artifacts that StateEye produces. This includes all of the outputs resulting from crawl sessions such as DOM snapshots, screenshots, state representations, state graph models, fragment classification results, generated test suites, and test execution summaries. The data layer is designed to be passive, with the only access being provided through the persistent layer (write/read)

2. Persistent Layer

Artifact Manager: Provides an abstraction layer between the file system and the business logic. Artifacts stored in the file system include: DOM files, screenshots, state graphs (JSON), fragment classification outputs, generated tests, execution results, etc.

Session Store: Contains metadata about each run (crawl) session including: Session Ids, Target URLs, Testing Modes, Execution Timestamp, List of Artifacts associated with the session to a provide means to track, reuse, and compare multiple testing sessions..

3. Business Logic Layer

- The Exploration Controller is responsible for coordinating both automated and manual exploration through the triggering of interactivity within the web application and the collection of results from the browser automation engine.
- The State Graph Builder is responsible for constructing and updating the application state graph by logging discovered states and transitions within the application during the course of exploration.
- Fragment Analyzer is responsible for breaking down each web state into smaller functional user interface (UI) fragments to allow for precise comparison and abstraction of the web state.
- The FRAGGEN Classifier applies fragment based equivalence logic to identify clone and near clone states as well as to locate data-fluid fragments; thus reducing overlapping exploration and false alarms from the regression tests.
- The Test Generator creates repeatable regression test cases from the inferred state graph by converting the exploration paths into executable test scripts.
- The Result Analyzer executes the generated regression test cases, applies fragment based test oracles, and evaluates the results of each test (success, failure, or warning) based on the execution logs generated during the execution of the tests. The results of the testing and analysis are forwarded to the presentation layer for visual representation.

4. Presentation Layer

- **UI:** The primary means through which developers and testers interact with the testing app. It allows for the entry of a URL for the application being tested, selection of the testing modes, and initiation of an exploratory test.
- **Manual Assisted Testing View:** The manual test is designed to assist testers in exploring the web application. It is designed to provide visual cues indicating the explored areas of the web application and the untested areas.
- **Coverage and Guidance View:** Displays the testing coverage achieved during testing, results of near-duplicate detection, warnings, and regression feedback to help testers make good decisions on how to test.

7.3 Top-Level Components

This overall architectural layout shows the components within the top level.

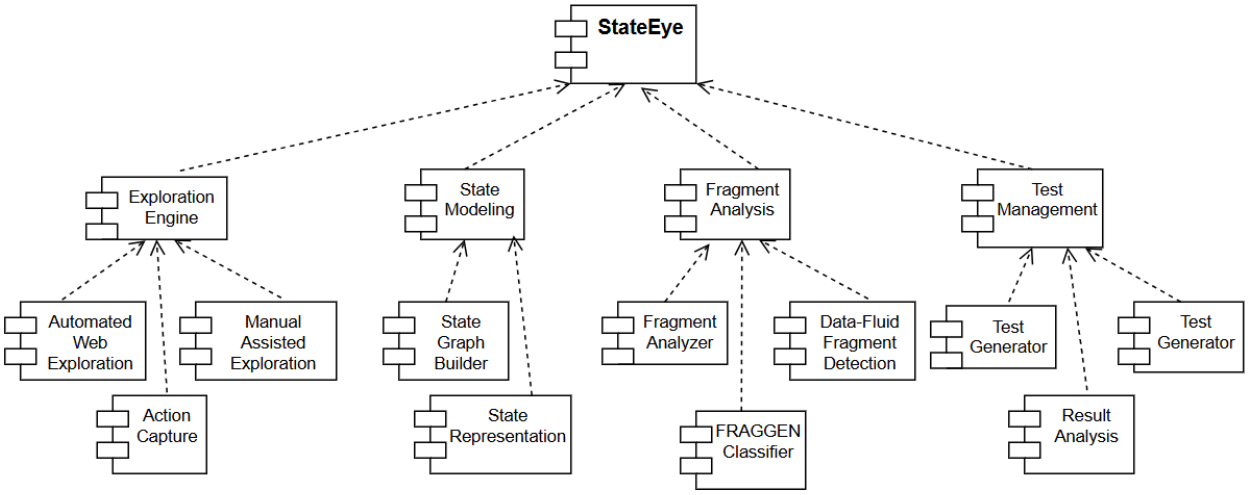


Fig-10: Top-Level Components

7.4 Deployment Diagram

A clearly defined and simple deployment architecture is essential to successfully deploy the StateEye application in a scalable, maintainable manner and to aid in implementation. The deployment diagram below has been easily partitioned into Front-End and Back-End with External Services.

The StateEye deployment diagram is given below:

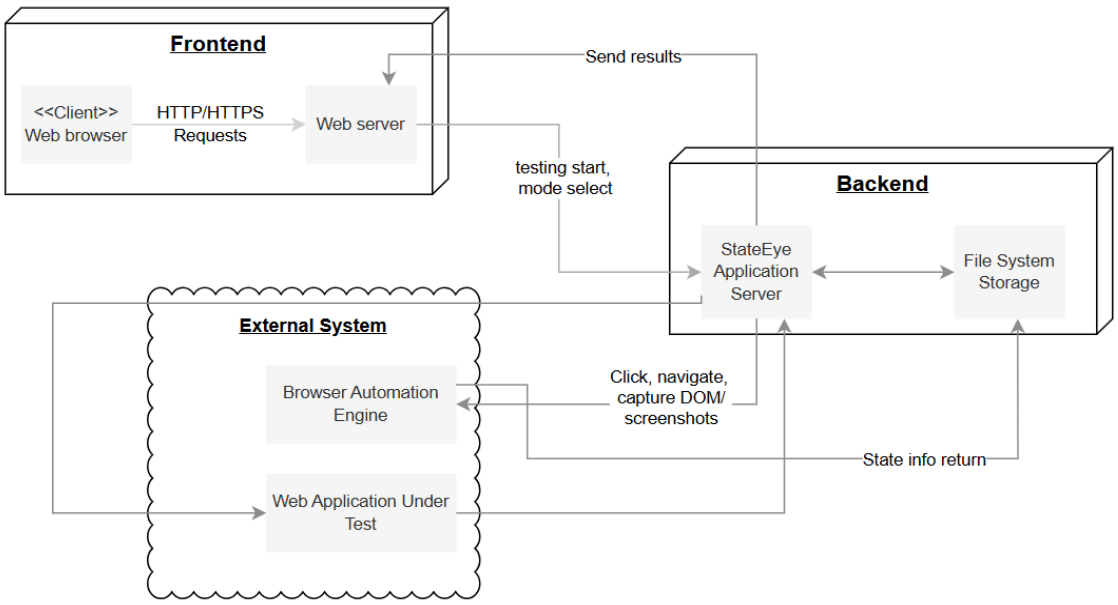


Fig-11: Deployment diagram of StateEye

8. Preliminary Test Plan

The purpose of this chapter is to give a brief overview regarding all of the major testing objectives as well as the features to be tested.

8.1 Overview of testing objectives

There are several major objectives for the testing of the StateEye system:

- Verify that StateEye fully meets both the functional and non-functional requirements for manual and automated web testing.

- Verify that the system correctly discovers web applications and produces valid state graphs.

- Verify that fragment-based state analysis is able to accurately identify clone and near-duplicate states.

- Verify that both automated and manual testing modes of the system will operate reliably and consistently.

- Verify that the generated regression test cases actually reflect the observed behavior of the application.

- Verify that fragment-based test oracles will identify functional regressions while tolerating cosmetic UI changes.

- Verify ease of use of coverage visualization and guidance tools to the testers.

- Verify that the system will be stable and perform well during large scale or long running crawls.

8.2 Test Cases

The following is a list of test cases that have been created to thoroughly evaluate the functions of the StateEye system.

Test Case 1: Provide Application URL

Test Case Scenario

A tester will provide a valid web application URL in order to begin an initial testing session.

Steps to Complete

- Enter an appropriate web application URL into the StateEye interface.
- Start the initial testing session.

Test Case 2: Selecting Modes
Test Case 2.1: Automated Test Selection

Test Scenario: After selecting automated web exploration mode, the tester will perform the following:

Steps: Provide a valid application URL; Click on "Automated Testing Mode;" Begin exploring the application.

Expected Results: StateEye will explore the application automatically; The application states and transitions will be identified without manual interaction.

Test Case 2.2: Manual Assisted Test Selection

Test Scenario: After selecting the manual assisted exploration mode, the tester will perform the following:

Steps: Provide a valid application URL; Click on "Manual Assisted Testing Mode;" And manually interact with the application.

Expected Results: The Tester's actions will be logged and tracked; The tested and untested areas will be flagged in real-time.

Test Case 3: State Graph Construction

Test Scenario: During exploration, StateEye constructs a state graph.

Steps: The user will conduct automated testing or manual testing, The user will observe state discovery and transition.

Expected Results: The states will be represented as nodes; The transitions will be recorded correctly on the state graph.

Test Case 4: Fragment-Based Analysis of Application States

Test Scenario: The system will analyze discovered application state fragments.

Test Steps: Explore numerous application states completely. Then, initiate fragment-level analysis.

Expected Result(s): States will be decomposed into individual UI fragments. Fragment representations will be correctly generated.

Test Case 5: Near-Duplicate and Clone State Detection

Test Scenario: StateEye will identify visually-clustered but structurally-discrete states

Test Steps: Explore application pages with similarly-structured layouts, but with differing data; then, perform the classification of fragments.

Expected Result(s): Application states will be classified as nearest-duplicate or clone states and thus reduce redundancy in exploration.

Test Case 6: Test Case Generation

Test Scenario: Automatically generate regression test cases.

Test Steps: Completely explore test states and build a test state graph; initiate test case creation

Expected Result(s): All test cases generated will be executable as they accurately represent valid user interactions.

Test Case 7: Evaluate Execution based on testing and Evaluate Results

Test Scenario: Rerun test cases created to find regressions.

Process: Execute test case set created, Evaluate test results.

Expected outcome:

- Accurate recording of results.
- Properly identify test failures and warnings.

Test Case 8: Verify Test Oracle using fragment-based information.

Test Scenario: The system will allow for cosmetic change of the UI, but will continue to detect functional regressions.

Process: Make cosmetic changes to UI layout or style, but don't change the way thing work. Re-run regression tests to verify that the tests still pass.

Expected Outcome:

- The UI will look different, but tests will still pass.
- The functional regression tests will still pass.

Test Case 9: Enforce storing sessions and storing artifacts for managing multiple crawlers

Test Scenario: Sessions created by the tests will be managed and stored correctly.

Process: Create multiple tests for the same application. Retrieve stored artifacts of the sessions created.

Expected Outcome:

- Each session will have its own independent storage of artifacts
- Each session will have its artifacts retrievable for evaluation and comparison.

Test Case 10: Determines whether the system can handle large application with many states

Test Scenario: The system can handle a large application with many states.

Process: Test large complex web application which has many pages and many ways to interact. Monitor the amount of pages crawled and how each/what pages are crawled.

Expected Outcome:

- The system should remain stable while crawling for a long duration.
- There is not a significant reduction in performance over ti

Test Case 11: Manual Assisted Exploration Guidance Accuracy

Test Scenario: The manual testing assistance is effective and helpful.

Steps: Use manual assistance to perform exploratory testing, View recommended next steps to take.

Expected Outcome:

- The assistance provided correctly reflects the actual unexplored states.

- The tester is efficiently directed to areas that have not been tested yet.

9. User Interface Design & User Manual

The design of a user interface provides an efficient means of communication between humans and computers. A set of interface design principles is followed to identify the objects and actions on the interface; then, a layout for a user interface prototype is created from the screen layout created in accordance with the identified objects and actions.

9.1 User Story:

Traditional web regression testing tools have a method for comparing full pages together and generating a screenshot of each page, performing a diff against some baseline images, and then reporting back to users any changes found. While this may seem reasonable on the surface, when users take into consideration the amount of duplication that would occur across many pages, they will see why this method cannot be applied.

Let’s say that there are nine pages on the site. The same navigation bar appears on every page of the site, the same header appears on every page of the site, and the same footer appears on every page of the site. In the situation above, a full-page testing tool would treat each of the nine pages as a completely independent unit from the other eight pages. It would take a screenshot of the navigation bar for each of those nine pages and do a diff against that set of nine in order to decide if the item had changed; it would use each of those same navigation bars to build nine

reports. If there is no difference between each of the nine navigation bars, a tester will be required to verify the existence of nine duplicate reports.

Now take that sample Web application and scale it to 50, 100, 500 pages; the amount of repetitive verification becomes exponentially larger than in the first example. As a result, testers waste valuable time verifying that similar components of the system have not changed, which could be better used to identify and test those components of the system that do, in fact, demonstrate differences.

9.1.1 Automated Mode - How it helps the tester

Steps to use StateEye in automation mode:

1. Provide URL/HTML for the website and click Start
2. StateEye will visit all pages in the time you configure in StateEye
3. StateEye will extract all states from those pages
4. The classification results will be:
   a. **Unique:** X states
   b. **Clone:** Y states (same nav/header/footer across pages)
   c. **Near Duplicate:** Z states (similar but with different data)
5. T Test Cases will be generated by StateEye (Y Clones will be skipped)
6. The Tester will execute the generated_tests.py file, T execution focused tests
7. All tests will pass. The Tester will validate the site with less work.

Developer made an edit to navigation:

1. The Tester will re-run the generated_tests.py file
2. The 1 Test for the nav state will FAIL (the visual regression will be found)
3. No noise will be generated from the 8 skipped clones
4. The Tester now knows exactly what changed and where to look for the change

Later a developer broke a product card template accidentally:

1. The Tester will re-run the generated_tests.py file
2. All near duplicate product cards will FAIL
3. Since they are all classified as near duplicate and share the same template, the Tester will know the root cause of the issue is 1 template change, not 7 separate pages' broken templates (as reported by a traditional full-page testing tool).

This demonstrates the power of fragment-based regression testing: work smart, not hard.

9.1.2 Manual Mode - How it helps the tester

While many of the tests we perform can be automated, not all can be automated. Some applications need people with conclusive judgement to perform certain functions, such as testing workflows, confirming business logics, or looking at edge cases; StateEye allows for this functionality through its manual mode.

Step 1: Launch: The tester launches the browser by typing in either the path to a local HTML file, the path to a directory or a URL and hitting "Start". At that point, StateEye will launch the browser and navigate directly to the specified site.

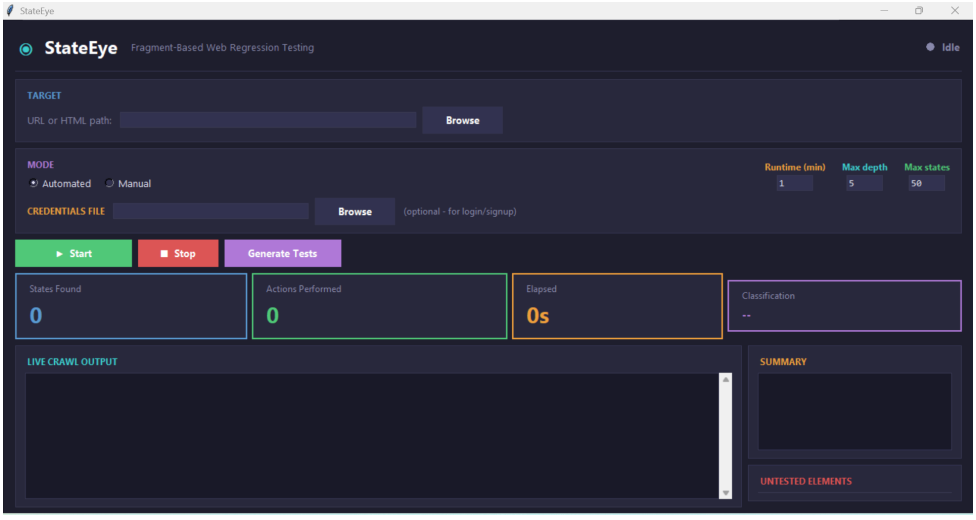
Step 2: Live Element Tracking: As the site begins to load in each place, StateEye will detect in real time all interactive states of the site: hyperlinks, buttons, forms, dropdowns, etc. All interactive states will be listed in the "Untested States" pane (in red).

Step 3: Explore to Test: As the tester explores/uses the website, StateEye will capture each click the tester makes in real time. When a tester clicks an interactive state, it will be marked as "testing" - the tester clicked on it, but hasn't explored the entire page yet.

Step 4: Complete on Page Level: Once a tester has clicked on every interactive state on a page, they will move the item from Untested (Red) to Tested (Green) - this confirms all interactive states have been travelled. A page is not tested until all interactive states have been exercised.

9.2 User manual

StateEye is a desktop tool built with Python and Tkinter. The full source is run locally.



9.2.1 Installation & Set up

Prerequisites

- Python 3.11+
- pip

Steps:

```
# 1. Clone the repository
git clone https://github.com/Trina-SE/StateEye.git
cd StateEye

# 2. Create and activate a virtual environment
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS/Linux
source .venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Install the Playwright browser binary (one-time)

python -m playwright install chromium

# 5. Launch the GUI

python pythonCode/stateeye_gui.py
```

9.2.3 Using StateEye (Automated mode)

1. Enter the target URL in the Target field:

- Web application: http://localhost:8080
- Local files: C:\sites\demo\index.html

2. Select Mode

Choose **Automated** mode from the mode selector.

3. Configure Crawl Parameters

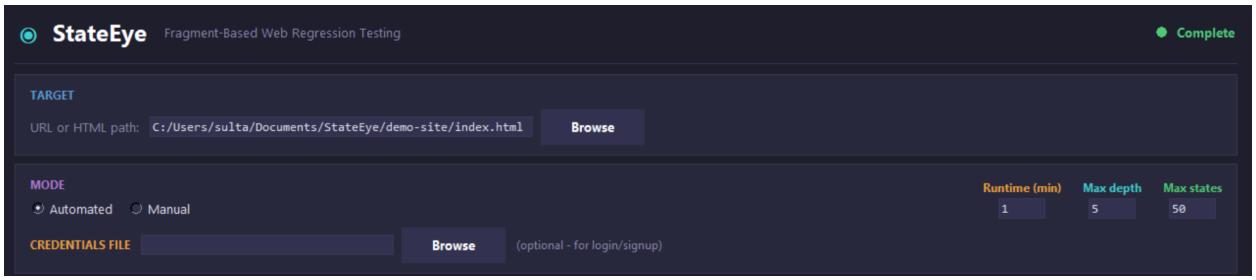
Adjust the following settings as needed:

Parameter	Description	Default
Runtime	Maximum crawl duration	60 seconds
Max Depth	Maximum navigation depth from starting page	5 levels
Max States	Maximum number of unique states to extract	50 states

4. Configure Authentication (Optional)

If the target site requires login:

1. Click **Browse** next to **Credentials File**
2. Select your credentials file
3. Ensure the file contains valid authentication data

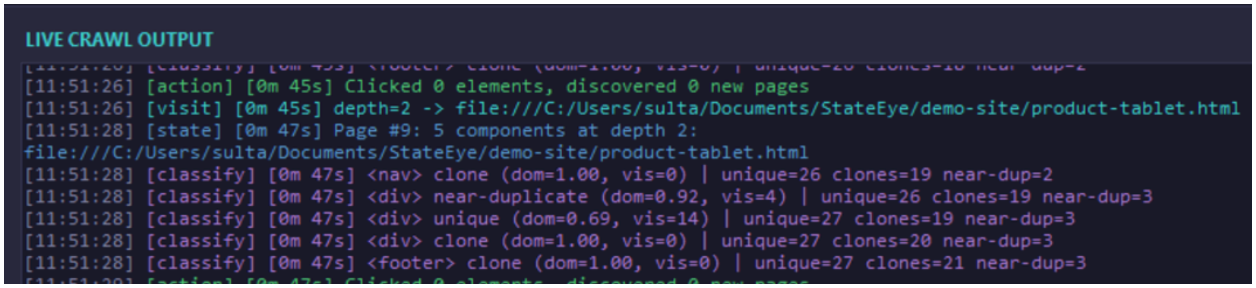


5. Start Crawl

Click **Start** to begin the automated crawl.

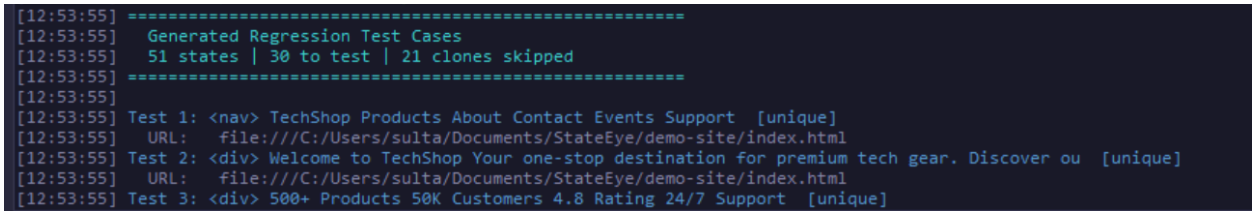
6. Output

- Real-Time Log Output

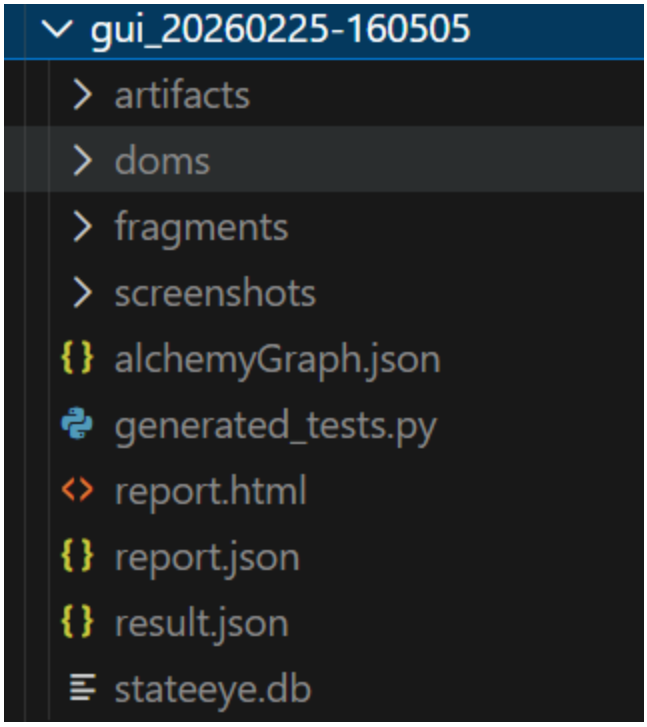


- Live Statistics

- Post-Crawl Actions: Generating Test Suit, Wait for the crawl to complete then Click Generate Tests button -> Test suite generation begins automatically -> Results are saved to the out/ directory



- Output Structure



9.2.4 Using StateEye (Manual mode)

Use manual mode when we need human judgment.

1. Enter the target URL in the Target field:

- Web application: http://localhost:8080
- Local files: C:\sites\demo\index.html

2. Select Mode

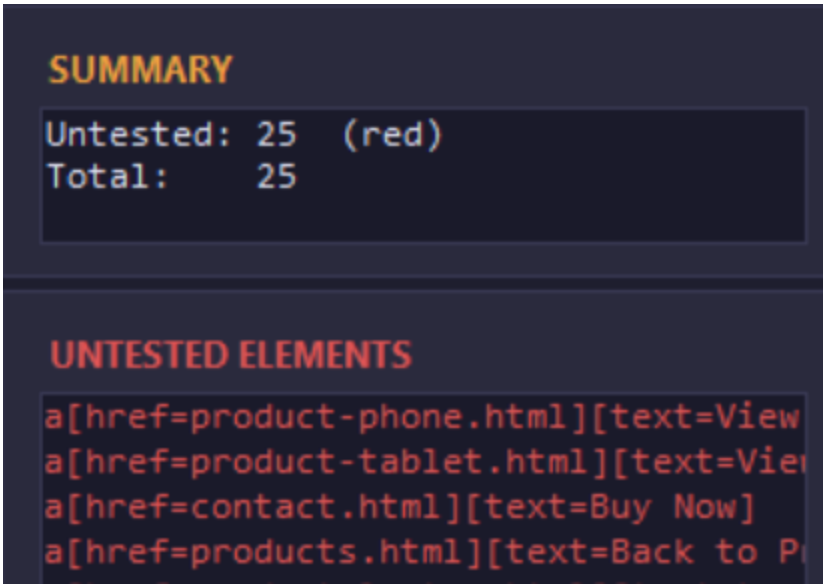
Choose **Manual** mode from the mode selector.

3. Start Crawl

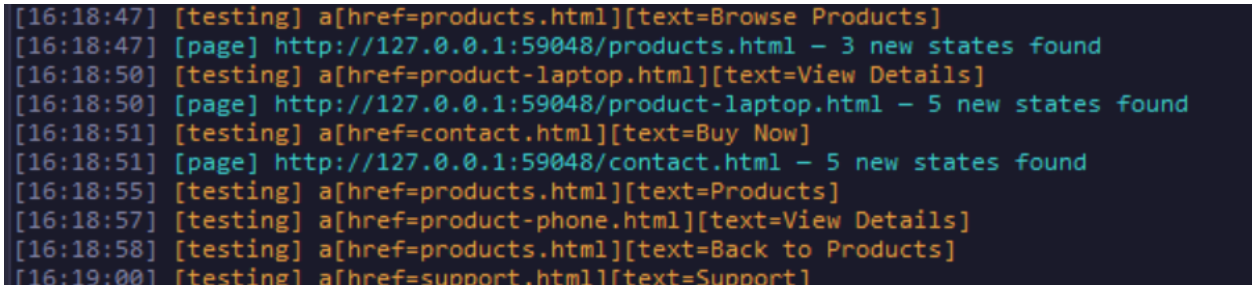
Click **Start** to begin the manual crawl.

4. Output

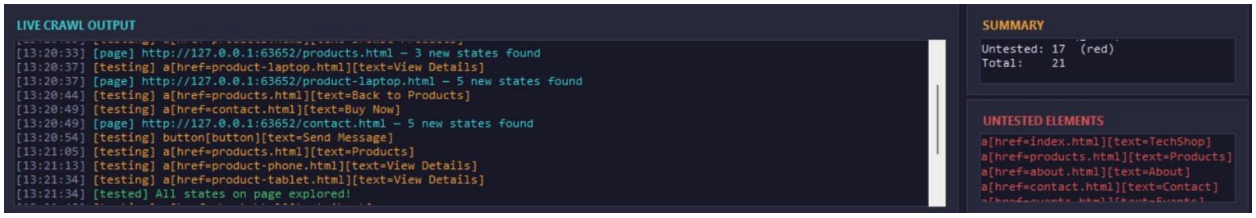
- Real-Time Log Output: StateEye continuously scans the page for interactive elements. All detected elements appear in the **Untested** panel (highlighted in red).



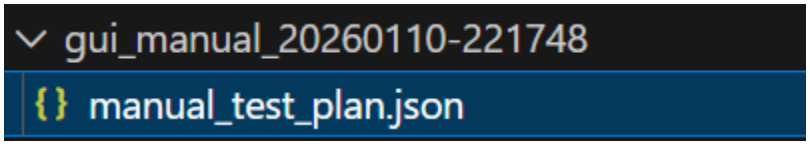
- As we interact with the site manually, StateEye tracks each action in real-time.



- Elements automatically move from **Untested** to **Tested** as we explore the element completely.
- When all interactive elements on a page have been tested, StateEye confirms: All states on page explored!



- Click **Generate Tests** to save your testing session



10. Conclusion

This paper provides the necessary technical information for developing a state-based web testing tool (StateEye) that can be used to test web applications by dividing them into fragments based on their similarity to one another. In addition to the technical details, this report includes information about the scope and limitations of the project, expected usage scenarios, and required system resources in order to provide an understanding of the overall complexity of the problem as well as the proposed solution. The analysis of the structural, behavioral, and responsibility issues of the components of the system was performed using scenario-based modeling, data-based modeling, and class-based modeling.

The architectural design of the StateEye system is described in detail. The architecture consists of a multi-layered approach including some components located at the core of the architecture, the architectural relationship between the components is diagrammed and described, including the exploration controller, the state graph builder, the fragment analysis engine and test generation engine. The report also provides a detailed view of how StateEye is architecturally related to its external systems (i.e., the web application to be tested and the web browser automation engine) by providing a detailed view of the state-eye's deployment in an architectural diagram.

A draft of a test plan has been provided that outlines testing objectives and an extensive array of test cases that will be used to ensure the functional correctness, robustness and usability of the system being tested. These test cases comprise automated and manual-assisted exploration, fragment-based state equality, the generation of regression tests and the visualisation of coverage. All diagrams, tables and structured descriptions used throughout this document serve to assist in improving the clarity and the ease with which the technical aspects of StateEye can be understood.

In addition to providing an effective means to reduce duplication in testing and to identify significant regressions in changing web applications, StateEye has many areas that could be addressed for further enhancements. These enhancements may consist of support for new browser automation frameworks, advanced fragment comparison techniques, scalability improvements to support large application environments and improved visualisation techniques to aid testers in completing their work.

Overall, this document provides an overview of the design, architecture and testing strategy for StateEye. In addition, this document is anticipated to provide a reference for developers, testers and stakeholders who are involved in developing and maintaining StateEye.

References

[1] S. Thummalapenta, K. V. Lakshmi, S. Sinha, N. Sinha, and S. Chandra, “Guided test generation for web applications,” in *Proceedings of the 35th International Conference on Software Engineering (ICSE)*, May 2013, pp. 162–171.

[2] IEEE Xplore, “Guided test generation for web applications.” [Online]. Available: https://ieeexplore.ieee.org/document/9765776

[3] Near-duplicate detection in web app model inference | Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering

[4] D. Roest, A. Mesbah, and A. van Deursen, “Regression testing Ajax applications: Coping with dynamism,” in *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*, 2010, pp. 127–136.

[5] A. M. Memon and M. L. Soffa, “Regression testing of GUIs,” *SIGSOFT Software Engineering Notes*, vol. 28, no. 5, pp. 118–127, Sep. 2003. [Online]. Available: https://doi.org/10.1145/949952.940088

[6] M. Grechanik, Q. Xie, and C. Fu, “Maintaining and evolving GUI-directed test scripts,” in *Proceedings of the 31st International Conference on Software Engineering (ICSE)*, IEEE, 2009, pp. 408–418.

[7] L. Christophe, R. Stevens, C. D. Roover, and W. D. Meuter, “Prevalence and maintenance of automated functional tests for web applications,” in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2014, pp. 141–150.

[8] M. Biagiola, A. Stocco, F. Ricca, and P. Tonella, “Diversity-based web test generation,” in *Proceedings of the 27th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019, pp. 142–153. [Online]. Available: https://doi.org/10.1145/3338906.3338970

[9] A. Mesbah, A. van Deursen, and D. Roest, “Invariant-based automatic testing of modern web applications,” *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 35–53, 2012.

[10] S. Mirshokraie and A. Mesbah, “JSART: JavaScript assertion-based regression testing,” in *Proceedings of the International Conference on Web Engineering (ICWE)*, 2012, pp. 238–252.

[11] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC)*, 2002, pp. 380–388. [Online]. Available: http://doi.acm.org/10.1145/509907.509965

[12] A. Milani Fard and A. Mesbah, “Feedback-directed exploration of web applications to derive test models,” in *Proceedings of the International Symposium on Software Reliability Engineering (ISSRE)*, IEEE, 2013, pp. 278–287.

[13] Y. Zheng, Y. Liu, X. Xie, Y. Liu, L. Ma, J. Hao, and Y. Liu, “Automatic web testing using curiosity-driven reinforcement learning,” in *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 423–435.

[14] Fragment-Based Test Generation for Web Apps: IEEE Transactions on Software Engineering, https://ieeexplore.ieee.org/document/9765776 , The approach named “FRAGGEN” https://arxiv.org/abs/2110.14043

[15] U. Praphamontripong, J. Offutt, L. Deng, and J. Gu, “An experimental evaluation of web mutation operators,” in 2016 IEEE Ninth International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2016, pp. 102–111